

AEGIS — Setup Checklist

Personal environment setup tasks for building the MVP/POC. Complete these in order — each section unblocks the next.

1. Local Development Environment

- ☐ Install Go 1.23+ (`brew install go`)
- ☐ Install Terraform 1.5+ (`brew install terraform`)
- ☐ Verify Node.js 20+ and npm are installed (`node --version`)
- ☐ Install Docker Desktop (for building/testing containers locally)
- ☐ Clone the AEGIS repo and verify the monorepo structure

2. GCP Project Setup

- ☐ Create a new GCP project (e.g., `aegis-dev`) in your personal GCP account
- ☐ Link a billing account to the project (free tier covers most dev usage)
- ☐ Install the gcloud CLI (`brew install google-cloud-sdk`)
- ☐ Authenticate: `gcloud auth login` and `gcloud auth application-default login`
- ☐ Set default project: `gcloud config set project aegis-dev`

3. Terraform — Project Bootstrap

- ☐ Copy `terraform/project/terraform.tfvars.example` to `terraform/project/terraform.tfvars`
- ☐ Fill in your `project_id` , `region` , and `billing_account`
- ☐ Run `terraform init` in `terraform/project/`
- ☐ Run `terraform plan` and review the output
- ☐ Run `terraform apply` to enable all required GCP APIs
- ☐ Verify APIs are enabled: `gcloud services list --enabled`

4. Terraform — Infrastructure

- ☐ Copy `terraform/infra/terraform.tfvars.example` to `terraform/infra/terraform.tfvars`
- ☐ Fill in your `project_id` and `region`
- ☐ Run `terraform init` in `terraform/infra/`
- ☐ Run `terraform plan` and review
- ☐ Run `terraform apply` to create Healthcare API DICOM stores, GCS bucket, Pub/Sub
- ☐ Verify DICOM store exists: `gcloud healthcare dicom-stores list --dataset=aegis --location=us-central1`
- ☐ Verify staging bucket exists: `gsutil ls`

5. Sample DICOM Data for Local Testing

- ☐ Download sample brain MRI DICOM files for testing (options below):
 - TCIA (The Cancer Imaging Archive): <https://www.cancerimagingarchive.net/>

- dicom.offis.de sample files: <https://dicom.offis.de/dcmtdk/>
- NEMA WG-6 sample DICOM: <ftp://medical.nema.org/medical/dicom/DataSets/>
- ☐ Place sample files in a local test directory (not committed to git)
- ☐ Verify files open in a DICOM viewer (e.g., Horos, 3D Slicer) to confirm validity

6. Frontend — Upload Portal

- ☐ `cd frontend/upload-portal && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3000>
- ☐ Install DICOM libraries: `npm install dcmjs dicom-parser`
- ☐ Test parsing a sample DICOM file in the browser console

7. Frontend — Admin Dashboard

- ☐ `cd frontend/admin-dashboard && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3001>
- ☐ Click **View** on any study row → OHIF Viewer iframe appears inline
- ☐ Click **Open in new tab** → viewer opens in a new browser tab
- ☐ Click **Audit Log** tab → shows event table (empty until actions are taken)
- ☐ For a defaced head study: click **Review defacing** → side-by-side OHIF panel (Before/After)
- ☐ Click **Routing** tab → Destinations and Rules sections load (empty state)
- ☐ Click **Institutions** tab → Institutions table loads (empty state)

7a. Routing Rules Engine

- ☐ Open admin dashboard → **Routing** tab
- ☐ Create a Destination (type: `dicomweb` , any URL)
- ☐ Create a Routing Rule (e.g. `modality=MRI` → `auto_approve`) → appears in priority-ordered table
- ☐ Upload an MRI study via Upload Portal → study status auto-set to `approved`
- ☐ Verify routing log: `GET http://localhost:8080/api/studies/{id}/routing-log`
- ☐ Disable/enable rule toggle works; delete cleans up

7b. Institution Management

- ☐ Open admin dashboard → **Institutions** tab
- ☐ Create an institution (type: `sender`)
- ☐ Click **Projects** button on the row → inline project-link panel expands
- ☐ Link institution to a project: enter project UUID from `GET /api/projects` , choose role
- ☐ Verify link appears in table; unlink removes it
- ☐ Edit institution details; disable/enable toggle works

7c. Anonymization Profiles

- ☐ Open admin dashboard → **Profiles** tab
- ☐ Create a profile: name `Research` , project `default` , retained tags `PatientAge` , `StudyDate` → appears in table

- ☐ Click **Set default** → badge "default" appears on the row
- ☐ Upload a DICOM via the Upload Portal → check the de-identified tag diff: `PatientAge` and `StudyDate` should show action `K` (kept)
- ☐ Clear the default; re-upload → those tags are stripped again (normal Basic Profile)
- ☐ Edit / delete the profile

7d. Email Digest Subscriptions

- ☐ Start API with Mailpit enabled (`SMTP_HOST=localhost SMTP_PORT=1025 go run .`)
- ☐ Open admin dashboard → **Notifications** tab
- ☐ Create a subscription: email `test@example.com` , project `default` , frequency `weekly`
- ☐ In the DB, force a digest due: `UPDATE digest_subscriptions SET last_sent_at = now() - interval '8 days'`
- ☐ Restart the API → scheduler fires on startup → check Mailpit at <http://localhost:8025> for the digest email
- ☐ Verify subject: `AEGIS Weekly Summary – default – <date range>` , body has study counts, no PHI/UIDs
- ☐ Delete the subscription from the Notifications tab

7e. Admin Users

- ☐ Open admin dashboard → **Users** tab
- ☐ Create a user: email `admin@example.com` , name `Test Admin` , role `admin` → appears in table
- ☐ Edit → change role to `viewer` , add notes → save
- ☐ Disable the user → row greys out; re-enable
- ☐ Delete with confirm prompt
- ☐ Verify `admin_user.created` , `admin_user.updated` , `admin_user.deleted` appear in Audit Log tab

7f. Project Settings

- ☐ Open admin dashboard → **Projects** tab
- ☐ Click **+ New project** → enter name only (slug auto-generated) → create → appears in table
- ☐ Click **Edit** on the row → change description → save → description updates
- ☐ Try editing slug → warning hint is shown
- ☐ Verify `project.created` and `project.updated` in Audit Log tab
- ☐ `GET http://localhost:8080/api/projects/{id}` returns the updated project

7g. Upload Portal QoL

- ☐ Upload a folder of DICOMs via the Upload Portal
- ☐ During the uploading stage, verify the current filename appears below the progress bar
- ☐ Long filenames are truncated with `...` prefix (>48 chars)
- ☐ (Optional) Throttle network in DevTools mid-upload → verify retries up to 3x before error

7h. Burned-in PHI Detection

- ☐ Install Tesseract OCR: `brew install tesseract` (macOS) or `apt-get install tesseract-ocr` (Linux)
- ☐ Start the PHI detection service:

```
cd phi-detection && pip install -r requirements.txt
uvicorn app.main:app --port 8082
```

- ☐ Verify health: `curl http://localhost:8082/healthz` → should show `{"status":"ok","backend":"tesseract"}`
- ☐ Start the Go API with PHI detection enabled:

```
cd api && PHI_DETECTION_SERVICE_URL=http://localhost:8082 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_phi_scan` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows PHI Scan badge: **pending**
- ☐ Click **Scan for PHI** → badge changes to **scanning** → then **clean** or **flagged**
- ☐ Check Audit Log tab → `phi_scan.triggered` and `phi_scan.complete` entries appear
- ☐ Verify admin can still Approve a flagged study (flag is informational, not blocking)

7i. Batch Import CLI

- ☐ Build the CLI: `cd api && go build -o aegis-import ./cmd/import`
- ☐ Dry run: `./aegis-import --dir ../test-data/brain-mri --project default --dry-run`
- ☐ Verify dry run output shows study count, modality, body part for each study
- ☐ Full import: `./aegis-import --dir ../test-data/brain-mri --project default`
- ☐ Verify studies appear in admin dashboard with `source=internal`
- ☐ Verify `import.batch` entries in Audit Log tab
- ☐ Re-run the same import → should report duplicate errors (StudyInstanceUID unique constraint)
- ☐ Test API endpoint:

```
curl -X POST http://localhost:8080/api/import/batch \
-H "Content-Type: application/json" \
-d '{"dir":"/absolute/path/to/test-data/brain-mri","project_slug":"default","dry_run":true}'
```

7j. QC Automation Service

- ☐ Start the QC service:

```
cd qc-service && pip install -r requirements.txt
uvicorn app.main:app --port 8083
```

- ☐ Verify health: `curl http://localhost:8083/healthz` → should show `{"status":"ok","backend":"basic"}`

- ☐ Start the Go API with QC enabled:

```
cd api && QC_SERVICE_URL=http://localhost:8083 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_qc_check` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows QC badge: **pending**
- ☐ Click **Run QC** → badge changes to **checking** → then **pass**, **warn**, or **fail**
- ☐ Check Audit Log tab → `qc_check.triggered` and `qc_check.complete` entries appear
- ☐ Verify admin can still Approve a study with warn/fail QC status (status is informational)

7k. NifTI/BIDS Conversion Service

- ☐ Install dcm2niix: `brew install dcm2niix` (macOS) or `apt-get install dcm2niix` (Linux)
- ☐ Start the BIDS service:

```
cd bids-service && pip install -r requirements.txt
uvicorn app.main:app --port 8084
```

- ☐ Verify health: `curl http://localhost:8084/healthz` → should show `{"status":"ok","backend":"dcm2niix"}`
- ☐ Start the Go API with BIDS service enabled:

```
cd api && BIDS_SERVICE_URL=http://localhost:8084 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_bids_conversion` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows BIDS badge: **pending**
- ☐ Click **Convert to BIDS** → badge changes to **converting** → then **complete**
- ☐ Click **Download BIDS** → browser downloads a zip archive
- ☐ Unzip and verify BIDS structure: `dataset_description.json`, `participants.tsv`, `sub-*/*anat/*.nii.gz` + `*.json`
- ☐ Check Audit Log tab → `bids_conversion.triggered` and `bids_conversion.complete` entries appear

7l. Metadata Classification Service

- ☐ Start the classification service:

```
cd classification-service && pip install -r requirements.txt
uvicorn app.main:app --port 8085
```

- ☐ Verify health: `curl http://localhost:8085/healthz` → should show `{"status":"ok","backend":"heuristic"}`
- ☐ Start the Go API with classification service enabled:

```
cd api && CLASSIFICATION_SERVICE_URL=http://localhost:8085 go run .
```

- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_classification` (any modality)
- ☐ Upload a study with missing modality → study row shows Classification badge: **pending**
- ☐ Click **Classify** → badge changes to **classifying** → then **classified**
- ☐ Verify study modality and body_part updated from "-" to classified values
- ☐ Verify routing rules re-evaluated (e.g. a `require_defacing` rule for HEAD now fires)
- ☐ Check Audit Log tab → `classification.triggered` and `classification.complete` entries appear

7m. MRI Protocol Compliance Service

- ☐ Start the protocol service:

```
cd protocol-service && pip install -r requirements.txt
uvicorn app.main:app --port 8086
```

- ☐ Verify health: `curl http://localhost:8086/healthz` → should show `{"status":"ok","backend":"basic"}`
- ☐ Start the Go API with protocol service enabled:

```
cd api && PROTOCOL_SERVICE_URL=http://localhost:8086 go run .
```

- ☐ Open admin dashboard → **Protocol Templates** tab
- ☐ Create a template: name `ADNI4 T1w`, project `default`, manufacturer `SIEMENS`, sequence type `T1w_MPRAGE`, add rules (e.g. RepetitionTime target 2300, tolerance 5%, severity warning)
- ☐ Open admin dashboard → **Routing** tab → create a rule: action `require_protocol_check` (any modality)
- ☐ Upload a study via the Upload Portal → study row shows Protocol badge: **pending**
- ☐ Click **Check Protocol** → badge changes to **checking** → then **compliant**, **minor_deviations**, or **non_compliant**
- ☐ Check Audit Log tab → `protocol_check.triggered` and `protocol_check.complete` entries appear with per-parameter findings
- ☐ Verify admin can still Approve a non-compliant study (status is informational)
- ☐ Edit/delete the template from the Protocol Templates tab

7n. Authentication Middleware

Auth is disabled by default (`AUTH_ENABLED=false`) — all admin endpoints auto-authenticate as `dev@aegis.local` .

Dev mode (default)

- ☐ Start the API: `cd api && go run .`
- ☐ Verify auth identity: `curl -s http://localhost:8080/api/auth/me | jq .` → shows `dev@aegis.local`, role `admin`

- ☐ Verify public routes work without auth: `curl -s http://localhost:8080/healthz → ok`
- ☐ Verify admin routes work without auth headers: `curl -s http://localhost:8080/api/studies | jq .total`
- ☐ Check Audit Log tab → audit entries show `dev@aegis.local` as the actor (not `admin`)

Production mode (GCP IAP)

- ☐ Start API with auth enabled:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=iap go run .
```

- ☐ Verify unauthenticated request is blocked:

```
curl -s http://localhost:8080/api/studies | jq .
# → {"error":"missing GCP IAP authentication header"}
```

- ☐ Verify unknown user is rejected:

```
curl -s -H "X-Goog-Authenticated-User-Email:
accounts.google.com:unknown@test.com" \
http://localhost:8080/api/studies | jq .
# → {"error":"user not registered: unknown@test.com"}
```

- ☐ Register a user and verify access:

```
# First disable auth temporarily to create user
cd api && go run .
curl -s -X POST http://localhost:8080/api/admin-users \
-H "Content-Type: application/json" \
-d '{"email":"admin@test.com","name":"Test Admin","role":"admin"}'
# Restart with auth enabled
AUTH_ENABLED=true AUTH_PROVIDER=iap go run .
curl -s -H "X-Goog-Authenticated-User-Email:
accounts.google.com:admin@test.com" \
http://localhost:8080/api/studies | jq .total
# → 0 (or study count)
```

Production mode (Azure AD)

- ☐ Start API with Azure AD auth:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=azure go run .
```

- ☐ Verify Azure header auth works:

```
curl -s -H "X-MS-CLIENT-PRINCIPAL-NAME: admin@test.com" \
http://localhost:8080/api/studies | jq .total
```

Production mode (AWS ALB + Cognito)

- ☐ Start API with AWS ALB auth:

```
cd api && AUTH_ENABLED=true AUTH_PROVIDER=aws go run .
```

- ☐ Verify AWS ALB header auth works (simulate ALB-injected JWT):

```
# Create a base64url-encoded JWT payload with email claim
PAYLOAD=$(echo -n '{"email":"admin@test.com"}' | base64 | tr '+/' '-_' | tr -d '=')
JWT="header.${PAYLOAD}.signature"
curl -s -H "X-Amzn-Idc-Data: ${JWT}" \
  http://localhost:8080/api/studies | jq .total
```

AWS S3 Storage

- ☐ Start API with S3 storage (LocalStack for dev):

```
# Start LocalStack (S3-compatible)
docker run -p 4566:4566 localstack/localstack
# Create bucket
aws --endpoint-url=http://localhost:4566 s3 mb s3://aegis-dev
# Run API with S3 backend
cd api && STORAGE_MODE=s3 S3_BUCKET=aegis-dev S3_REGION=us-east-1
S3_ENDPOINT=http://localhost:4566 go run .
```

- ☐ Upload a study via the Upload Portal → verify files stored in S3 bucket
- ☐ View study in OHIF → verify DICOMweb proxy retrieves from S3
- ☐ Verify signed URLs work: upload + download flows complete without error

Azure AD App Registration Setup (for production Azure deployments)

1. Register the application in Azure Portal:

- Go to Azure Portal → Microsoft Entra ID → App registrations → New registration
- Name: AEGIS Admin Dashboard
- Supported account types: "Accounts in this organizational directory only" (Single tenant)
- Redirect URI: `https://<your-app-url>/.auth/login/aad/callback`
- Click Register

2. Configure authentication:

- Go to the app registration → Authentication
- Add platform: Web
- Redirect URIs: `https://<your-app-url>/.auth/login/aad/callback`
- Check "ID tokens" under Implicit grant

3. API permissions:

- Microsoft Graph → `User.Read` (delegated) — to read user email
- Grant admin consent for the directory

4. Configure the hosting platform:

- **Azure App Service:** Go to Settings → Authentication → Add identity provider → Microsoft → Select the app registration
- **Azure Application Gateway + AAD:** Configure AAD authentication on the gateway; it injects `X-MS-CLIENT-PRINCIPAL-NAME`
- **Self-hosted with MSAL.js:** Add MSAL.js to the frontend; backend validates JWT Bearer tokens (future enhancement)

5. Add authorized users:

- In the AEGIS admin dashboard Users tab, add each Azure AD user's email as an admin user
- Their Azure AD login email must match the `admin_users.email` field (case-insensitive)

6. Environment variables:

```
AUTH_ENABLED=true
AUTH_PROVIDER=azure # or "auto" to support both IAP and Azure
```

7o. RBAC Enforcement (Viewer Role)

The viewer role is read-only — viewers can browse all data but cannot create, update, delete, or trigger processing.

Backend verification

- ☐ Create a viewer user (with API running in default dev mode):

```
curl -s -X POST http://localhost:8080/api/admin-users \
-H "Content-Type: application/json" \
-d '{"email":"viewer@aegis.local","name":"Test Viewer","role":"viewer","enabled":true}'
```

- ☐ Restart API as viewer: `cd api && DEV_USER_EMAIL=viewer@aegis.local go run .`
- ☐ Verify read endpoints work:

```
curl -s http://localhost:8080/api/studies | jq .total # → 200 OK
curl -s http://localhost:8080/api/institutions | jq . # → 200 OK
curl -s http://localhost:8080/api/auth/me | jq .role # → "viewer"
```

- ☐ Verify write endpoints are blocked:

```
curl -s -X POST http://localhost:8080/api/projects \
-H "Content-Type: application/json" \
-d '{"name":"test"}' | jq .
# → {"error":"insufficient permissions: requires admin role"}
```

Frontend verification

- ☐ Open admin dashboard at <http://localhost:3001> as viewer
- ☐ Verify Users tab is NOT visible in navigation

- ☐ Verify Studies tab: View + Download BIDS visible; Approve/Reject/Share/processing buttons hidden
- ☐ Verify Routing tab: data visible; Add/Edit/Delete/Toggle buttons hidden
- ☐ Verify Institutions tab: data visible; Add/Edit/Delete/Toggle/Link/Unlink hidden; Projects view button visible
- ☐ Verify Profiles, Protocol Templates, Notifications, Projects tabs: data visible; create/edit/delete buttons hidden
- ☐ Switch back to admin: restart API with `DEV_USER_EMAIL=dev@aegis.local` (or default) — all buttons return

7p. Defacing Service Backends

The defacing service supports multiple pluggable backends selected via the `DEFACE_TOOL` env var. DeepDefacer (3D U-Net) is the recommended production default for speed.

Local verification (without Docker)

- ☐ Install dependencies:

```
cd defacing
pip install -r requirements.txt
pip install deepdefacer
```

- ☐ Start with DeepDefacer:

```
DEFACE_TOOL=deepdefacer uvicorn app.main:app --port 8081
```

- ☐ Verify health endpoint:

```
curl http://localhost:8081/healthz | python3 -m json.tool
# → {"status":"ok","backend":"deepdefacer","available":true}
```

- ☐ Test explicit backend selection:
 - `DEFACE_TOOL=nibabel` → health shows `nibabel-fallback`
 - `DEFACE_TOOL=mri_deface` → falls back to `nibabel-fallback` (unless `mri_deface` installed)
 - `DEFACE_TOOL=auto` → uses first available (deepdefacer if installed)

Docker verification

- ☐ Build with DeepDefacer:

```
docker compose build defacing
# docker-compose.yml sets INCLUDE_DEEPDEFACER=true by default
```

- ☐ Start defacing service:

```
docker compose up defacing
```

- ☐ Verify via API health: `curl http://localhost:8080/healthz | python3 -m json.tool` → `services.defacing` shows healthy

7q. Automated Processing Pipeline

The pipeline auto-dispatches processing services after upload. Enabled by default (`PIPELINE_AUTO=true`).

- ☐ Configure routing rules that require multiple services (e.g. `require_classification` + `require_defacing` + `require_qc_check` + `require_bids_conversion`)
- ☐ Upload a study via the upload portal
- ☐ Check API logs for `pipeline: dispatching classification for ...` — classification runs first
- ☐ After classification completes, verify logs show parallel dispatch of defacing + PHI scan (if configured)
- ☐ After defacing completes, verify QC check and BIDS conversion are auto-dispatched
- ☐ Verify audit log shows `pipeline.dispatch` entries for each service
- ☐ Test manual override: click a manual trigger button in the dashboard — should still work
- ☐ Test disable: set `PIPELINE_AUTO=false` , upload again — no auto-dispatch, manual buttons required

8. Go API

- ☐ `cd api && go run .` — verify health endpoint at <http://localhost:8080/healthz>
- ☐ Add GCP SDK dependencies: `go get cloud.google.com/go/storage`
`cloud.google.com/go/healthcare`
- ☐ Implement signed URL generation endpoint
- ☐ Test upload flow: browser → signed URL → GCS staging bucket

8a. Local Services (Docker Compose)

`docker-compose.yml` starts the **full platform stack** — database, email, viewer, Go API, and all Python sidecar services:

- ☐ `docker compose up -d` — builds and starts all 10 services
- ☐ Verify API health: `curl http://localhost:8080/healthz | python3 -m json.tool`
 - Should show `"status":"ok"` , `"database":"healthy"` , `"storage":"healthy"` , and all 6 sidecar services as `"healthy"`
- ☐ Verify OHIF loads at <http://localhost:3002> (shows the AEGIS data source)
- ☐ Verify Mailpit web UI at <http://localhost:8025>

Services started by `docker compose up` :

Service	Port	Notes
postgres	5432	Data in pgdata named volume (persists across restarts)
mailpit	1025 / 8025	SMTP capture + web UI
ohif	3002	OHIF Viewer (waits for API health)

api	8080	Go API (runs migrations on startup)
defacing	(internal)	Defacing service
phi-detection	(internal)	Burned-in PHI detection
qc-service	(internal)	QC automation
bids-service	(internal)	NIfTI/BIDS conversion
classification-service	(internal)	Metadata classification
protocol-service	(internal)	MRI protocol compliance

Start individual services:

```
docker compose up -d postgres      # just database
docker compose up -d api           # API + postgres (auto-dependency)
docker compose build               # rebuild all images after code changes
docker compose down -v            # stop + destroy volumes (fresh start)
```

Note: The frontends (upload-portal on :3000, admin-dashboard on :3001) still run via `npm run dev` outside Docker, proxying `/api` to `localhost:8080`.

8b. Email (Local Dev with Mailpit)

Email is disabled by default — all calls are silent no-ops when `SMTP_HOST` is unset. Mailpit is included in `docker-compose.yml` (step 8a). To run standalone:

Email is disabled by default — all calls are silent no-ops when `SMTP_HOST` is unset.

- ☐ Run Mailpit (or use `docker compose up -d mailpit`):

```
docker run -p 1025:1025 -p 8025:8025 axllent/mailpit
```

- ☐ Start the API with SMTP env vars:

```
cd api && SMTP_HOST=localhost SMTP_PORT=1025 go run .
```

- ☐ Open Mailpit web UI at <http://localhost:8025>
- ☐ Trigger each notification to verify:
 - Upload a study via the portal — enter your email in the "Your email (optional)" field on the preview screen
 - Approve the study in the admin dashboard → uploader gets approval email
 - Reject a study → uploader gets rejection email
 - Create a share for an approved study → recipient gets share email with download link
- ☐ Verify no crash when `SMTP_HOST` is unset (email silently skipped, no error returned)
- ☐ Verify upload with no email entered still completes successfully (no notification sent)

9. GitHub Repository

- ☐ Verify remote is set: `git remote -v`
- ☐ Push monorepo scaffold to `develop` branch
- ☐ Set up branch protection on `main` and `develop` (require PR reviews, prevent deletion)
 - **Requires GitHub Pro** (\$4/month) for private repos, or make the repo public
 - Go to Settings → Branches → Add branch protection rule
 - Branch name patterns: `main` and `develop`
 - Enable: "Require a pull request before merging", "Do not allow deletions"
- ☐ CI is configured: `.github/workflows/ci.yml` runs automatically on PRs to `develop` and `main`
 - Go build + vet, Python syntax check (6 services), TypeScript type check (3 apps), Docker build (7 images)
- ☐ Verify CI passes: open a test PR and check the Actions tab
- ☐ Run `make lint` locally to validate before pushing

10. Future — Before Proposing to Work

- ☐ Have a working end-to-end demo: upload → anonymize → view in OHIF
- ☐ Prepare a 5-minute screen recording of the demo flow
- ☐ Draft a one-page proposal covering: problem, solution, differentiation, cost estimate
- ☐ Identify potential pilot users / departments at your institution
- ☐ Research your company's internal startup / innovation program requirements

Generated 2026-02-18. Updated 2026-02-18. See `AEGIS_Architecture.md` for the full system design.